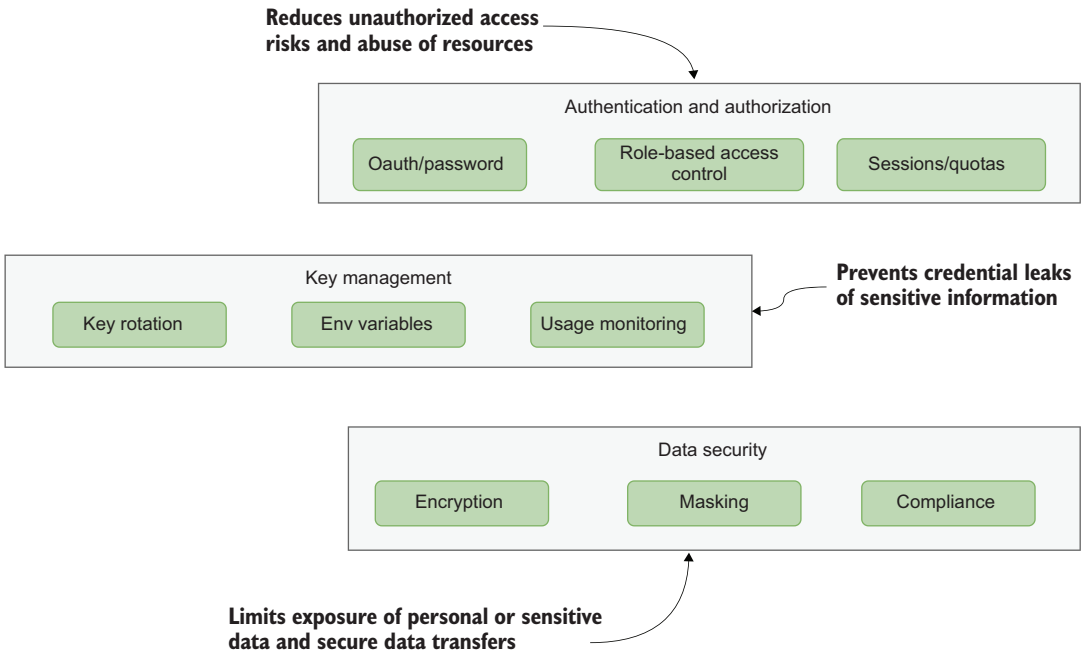


Figure 9.2 A layered approach covering secure development practices, authentication and authorization, API key management, data protection, and compliance to ensure robust security and threat prevention

For authentication, we have several options:

- *Username/password*—The simplest approach is to register users with a username and a password. While basic and widely recognized, handling passwords securely is complex, demanding a robust implementation of password policies that enforce sufficient length, complexity, and regular updates. This method is mostly not recommended in the current era due to the difficulty in implementing and maintaining user information securely.
- *OAuth 2.0/social login*—By using OAuth 2.0, users of an application can allow limited access to their resources on another site without exposing their credentials. Allowing users to authenticate using existing accounts such as Google or Facebook simplifies the login process and uses the robust security infrastructure of these established providers. With social login, it's crucial to carefully review the permissions being requested by the third-party application and avoid excessive permission requests; otherwise, users might not agree to share their data with your platform. This method is highly recommended if you provide mechanisms for users to unlink their social accounts and delete their data.
- *Passwordless authentication*—Passwordless authentication offers a modern approach to verifying users by using email or SMS verification codes, eliminating the need for them to create or remember passwords.